



CVR COLLEGE OF ENGINEERING

(An Autonomous Institution, NAAC Accredited and Affiliated to JNTU, Hyderabad)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M), Rangareddy (D), Telangana- 501 510

DEPARTMENT OF CSE (DATA SCIENCE)

B.Tech II Year II Semester

Real-Time/Field - Based Research Project Review - 0

Mind Mancer

By

B. Ram Ganesh(23B81A6730)

N. Rohith Reddy(23B81A6734)

D. Dhanush(23B81A6711)

Under the guidance of

Mrs. M. Nikitha

Designation

Associate Professor-CSIT



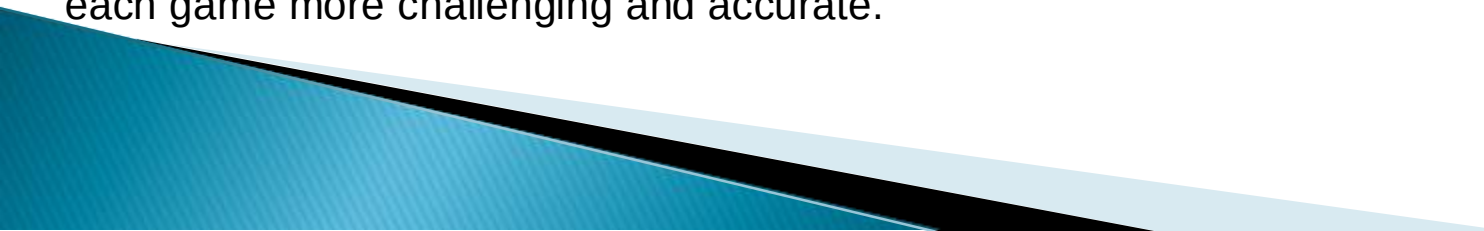
Abstract:

Mind Mancer is an AI-driven web application that simulates a mind-reading game by guessing a character, object, or concept the user is thinking of. The primary goal of this mini-project is to create an engaging and interactive game that uses artificial intelligence to predict the user's chosen entity through a series of yes/no questions. The application leverages a decision tree algorithm to narrow down possible options based on the user's responses, ultimately guessing the character or object the user is thinking of.

The game begins when the user thinks of a character or object, and the AI starts asking questions. The AI uses a pre-structured decision tree, which evolves over time as it learns from new interactions. It stores a knowledge base of different characters, objects, and their unique attributes, refining its guesses with each new game. Users also have the option to add new characters to the game, allowing the AI's knowledge base to expand dynamically.

The backend of Mind Mancer is powered by Python, using machine learning libraries and decision-tree algorithms to manage the questioning process and improve over time. The frontend is developed with HTML, CSS, and JavaScript, ensuring a seamless user interface that is both intuitive and interactive. To facilitate smooth gameplay, the game is hosted on a cloud platform with a robust database to store and manage character data.

Mind Mancer not only showcases the practical application of AI and machine learning in entertainment but also offers users a fun and evolving experience. As the game interacts with more players, the AI continually enhances its predictive capabilities, making each game more challenging and accurate.



Problem Statement:

- Decision Tree Construction and Optimization:**

How can we construct and optimize a decision tree that efficiently narrows down possibilities based on yes/no questions to improve the AI's accuracy in guessing the user's thought?

- Knowledge Base Expansion:**

How can we develop a dynamic system that allows users to add new characters or objects to the knowledge base, and how can we ensure that the new entries enhance the AI's decision-making process without affecting performance?

- Machine Learning for Adaptive Learning:**

How can we incorporate machine learning techniques to allow the AI to learn from user interactions over time, adapting its question-asking strategy and improving its guessing accuracy with each new game?

- Scalability of the System:**

How can we design the system to scale efficiently, handling a growing database of characters and objects without causing delays or slow performance as more users interact with the game?

- Interactive and User-Friendly Interface:**

How can we design an intuitive, responsive, and interactive user interface that provides a seamless experience for the user while maintaining the functionality of the underlying AI-driven game?



Existing works/methods and challenges:

▶ Existing works:

•Machine Learning-based Character Guessing:

Some projects have explored the use of machine learning algorithms such as decision trees, random forests, or neural networks to classify or guess objects based on user inputs. These methods help improve the accuracy and prediction rate over time by learning from historical interactions and continuously refining the model.

•Knowledge Graphs and Semantic Networks:

Some projects have used knowledge graphs to represent the relationships between various entities and their attributes. By mapping these relationships, an AI can more intelligently predict a user's thought based on contextual and semantic understanding, rather than just a simple decision tree.

▶ Challenges:

Handling Large and Complex Knowledge Base:

As the game grows and more characters or objects are added, the knowledge base can become increasingly complex. Managing and maintaining a large database while ensuring quick retrieval times for accurate guessing can be a significant challenge.

Machine Learning Model Training and Updating:

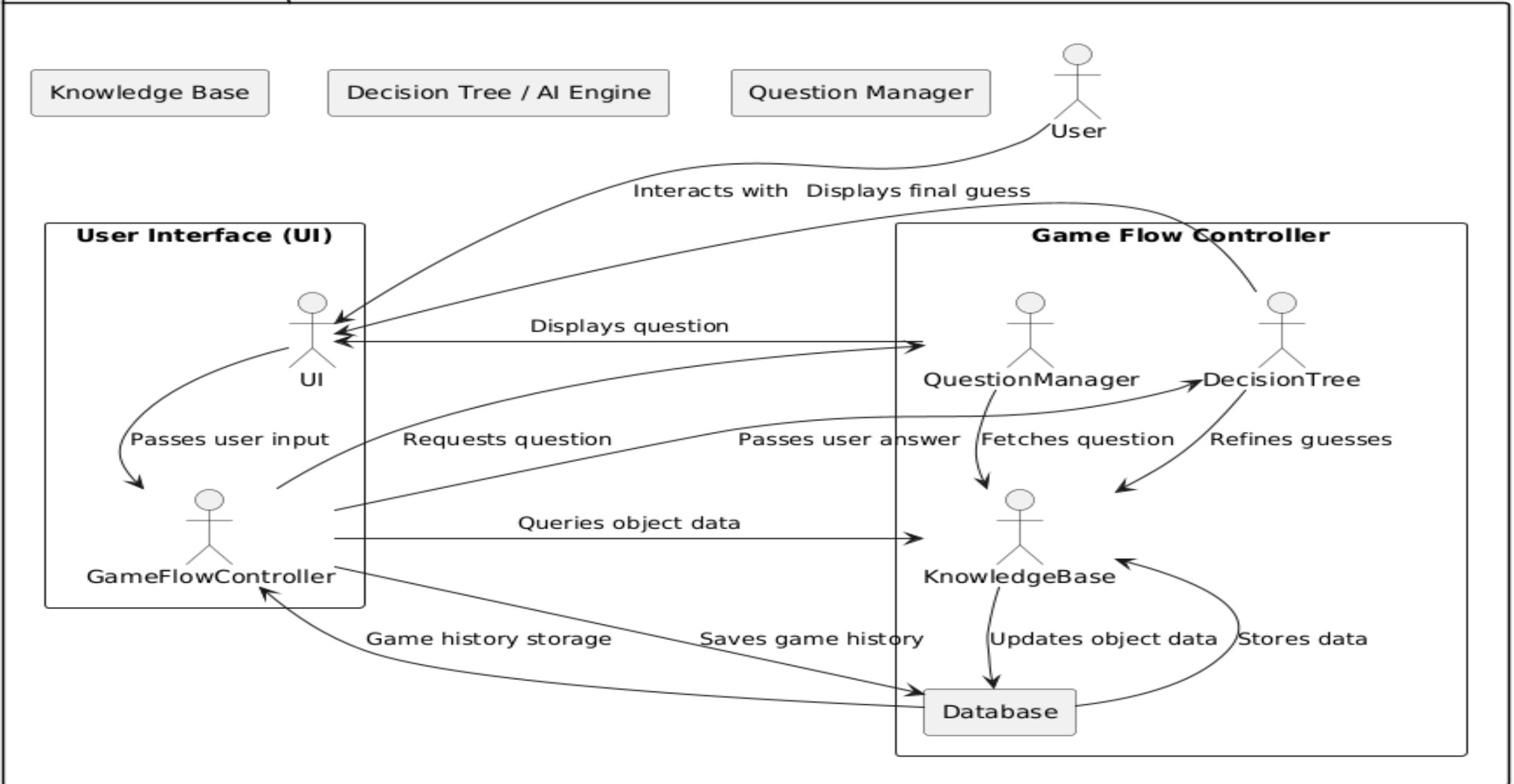
If machine learning is used, the challenge lies in training the model effectively, ensuring it learns from user interactions, and updating it in real-time without compromising system performance.

Proposed Method:

- ▶ **Description:** Mind Mancer is an AI-powered guessing game that uses decision trees and machine learning to predict characters, objects, or concepts a user is thinking of, based on yes/no questions. It continuously learns from user interactions and improves its guessing accuracy over time.
- ▶ **List of Modules/Features:**
 - **Decision Tree Algorithm:** Core AI logic to ask relevant questions and guess the user's thought.
 - **Knowledge Base Management:** Stores and manages character/object data, allowing dynamic updates.
 - **Machine Learning Integration:** Learns from user inputs to improve accuracy and question strategy.
- ▶ **Advantages:**
 - **Engaging Gameplay:** Interactive and fun, keeping users entertained with each session.
 - **Adaptive Learning:** AI improves over time, offering a more accurate and challenging experience.
 - **Dynamic Content:** Users can add new characters, expanding the knowledge base.

Architecture diagram:

Mind Mancer Game



Technology Stack:

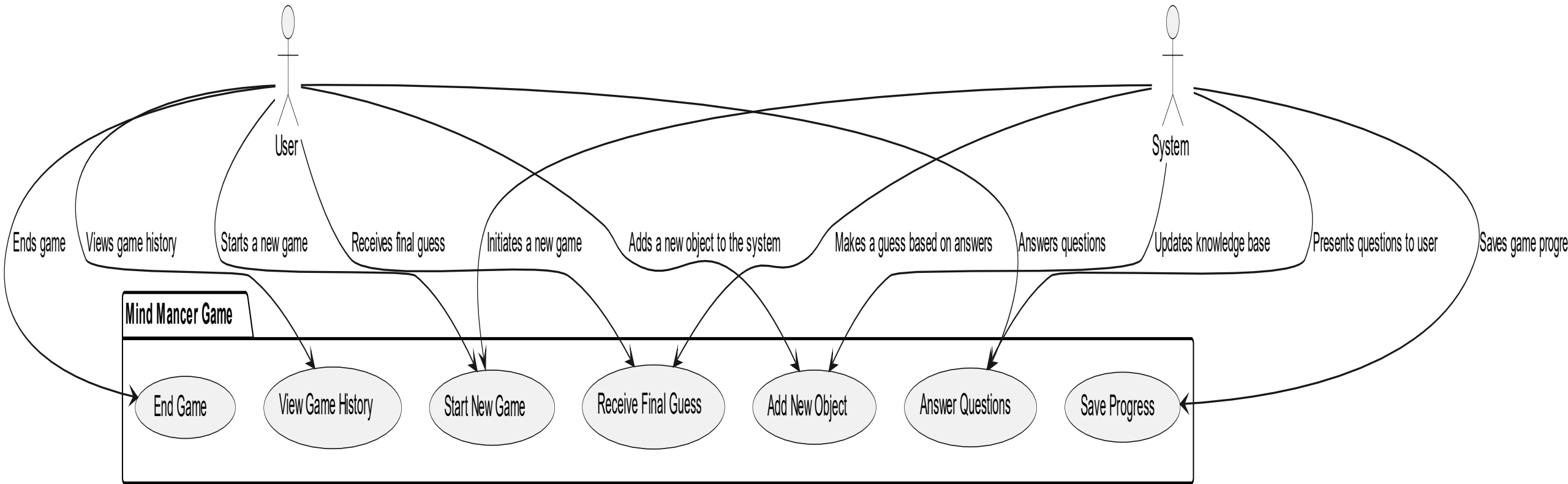
▶ Software Requirements:

- **Operating System:** Windows, macOS, or Linux.
- **Programming Languages:** Python, HTML, CSS, JavaScript.
- **Libraries/Modules:** NumPy, Pandas, Scikit-learn, Flask-SQLAlchemy
- **Development Environment:** Visual Studio Code, PyCharm, or WebStorm.
- **Additional Tools:** Docker, Postman, Figma, or Adobe XD.
-

▶ Hardware Requirements:

- - **Processor:** Dual-core CPU (Intel i3 or equivalent) or higher.
 - **RAM:** 4GB minimum (8GB recommended for smoother experience during use).
 - **Storage:** 10GB of free disk space (for browser cache and web assets).
 - **Graphics:** Integrated graphics (no dedicated GPU required).
- 

Use-Case/Data Flow Diagram:



References:

- **"Decision Trees" – GeeksforGeeks**

A comprehensive article explaining decision tree algorithms, which are used for classification and prediction.

<https://www.geeksforgeeks.org/decision-tree-algorithm/>

- **"Introduction to Machine Learning" – Scikit-learn Documentation**

Provides insights into machine learning algorithms, including decision trees and their applications in prediction.

<https://scikit-learn.org/stable/tutorial/basic/tutorial.html>

- **"The 20 Questions Game" – Wikipedia**

An article on the classic 20 Questions game, which is the conceptual foundation for games like "Mind Mancer".

[https://en.wikipedia.org/wiki/Twenty_Questions_\(game\)](https://en.wikipedia.org/wiki/Twenty_Questions_(game))

- **"Machine Learning Yearning" by Andrew Ng**

A well-regarded book by Andrew Ng that discusses the process of applying machine learning algorithms to real-world problems.

<https://www.deeplearning.ai/machine-learning-yearning/>



Thank You

